AULA 16

O caminho até o Nav2 navegar, e a sessão que se reproduz sem o robô

Etapa 8 · terça, 03/11/2026 · sala SJ205 · Prof. Dácio Moreira de Souza

Sistema que só funciona com você na frente não está entregue.

É isso que o item 2 do TP4 cobra ao pedir uma árvore de comportamento E uma sessão gravada que se reproduz offline: a árvore decide sem você, e o bag prova sem você.

Cenas dos últimos episódios: onde paramos

- O Nav2 subiu com o mapa de vocês, e a nuvem fechou depois do 2D Pose Estimate
- Antes disso: três nuvens de 6 a 8 cm, e uma delas 5,92 m fora do lugar
- A frase que ficou: concordância interna não é evidência
- E a aula começou com um defeito meu: rviz:=true, catorze processos, nenhum RViz2
- O launch_arguments do include não é escopado — e nada reclamou

O que você sai sabendo hoje

- Descobrir, em dez segundos, por que o SEU Nav2 não está navegando
- Mandar o robô até um objetivo e provar que ele chegou — não que ele planejou
- Escolher, e saber dizer, ONDE mora a decisão de interromper a navegação
- Medir uma sessão gravada com o robô desligado: a análise que não depende de sorte

E o que cada uma vale no SEU projeto

- Diagnosticar: você deixa de depender de mim para destravar — e trava menos
- Chegar ao objetivo: é a primeira vez que o seu robô faz algo sozinho, de ponta a ponta
- Onde mora a decisão: é arquitetura, e é o que você vai defender na arguição
- Medir sem o robô: a demonstração do fim do semestre não pode depender de sorte
- As datas de cada etapa estão na página e em "Gates de cada TP" — consulte, não decore

Como a aula funciona hoje

- Eu demonstro, no meu ambiente, o caminho que funciona — do sistema saudável ao goal
- Vocês observam e anotam o que vão procurar em casa: a linha da tela, o nome do nó
- Três casos ao vivo, os mais prováveis: eu quebro, o diagnóstico aponta, eu corrijo
- O que é para rodar no SEU ambiente está na tarefa da semana, com o caminho, em ordem
- Se o tempo apertar, o bloco da sessão gravada cede: ele está inteiro na página

Onde está o material

- A página da Aula 16: o que eu mostrar hoje, com os comandos, e a tarefa da semana
- Os slides em PDF, na mesma página
- aula16-clinica: o diagnóstico do Nav2, Python puro
- aula16-arvore-bag: as duas árvores e a régua da sessão gravada
- O bloco de download abre a página e o seu trabalho em casa — não é para agora
- Agora: olhe a tela e anote ONDE olhar. O comando você acha na página

BLOCO 1

O caminho para navegar, demonstrado

Terminal A: o sistema, com o RViz2

```
export PB_USER=dacioms
export PB_DIR="$HOME/projeto-pb-$PB_USER" && export PB_WS="$PB_DIR/ros2_ws"
source /opt/ros/humble/setup.bash
source "$PB_WS/install/setup.bash"
ros2 launch aula15_nav2 nav2.launch.py rviz:=true

# no RViz2: 2D Pose Estimate na posicao do robo
```

Terminal novo começa SEMPRE pelas variáveis e pelas duas linhas de source. Sem a segunda, o erro diz "pacote não encontrado" — e o pacote existe. Olhem a nuvem fechando depois do clique.

O goal, e a prova de que ele chegou

- No RViz2: 2D Goal Pose num ponto livre — ele publica em /goal_pose
- Olhem três coisas: o plano global até o alvo, a janela do costmap local, o robô andando
- O que prova é o robô CHEGANDO: 20 s de celular apontado para a tela, saindo e chegando
- Print de plano desenhado é planejamento, não navegação
- No seu projeto: um ponto do SEU mapa, escolhido sobre o costmap — passo 4 da tarefa

Terminal B: o diagnóstico, no sistema saudável

```
export PB_USER=dacioms
export PB_DIR="$HOME/projeto-pb-$PB_USER" && export PB_WS="$PB_DIR/ros2_ws"
source /opt/ros/humble/setup.bash
source "$PB_WS/install/setup.bash"
cd "$PB_DIR/aula16-clinica" && python3 diagnosticar_nav2.py

#  achei  /cmd_vel so' com os publicadores do Nav2
#         velocity_smoother + behavior_server x5, como esperado
#  Nao sobrou nada para olhar do que este script alcanca.
```

Dez verificações, na ordem em que compensa olhar. Saudável é isto: lista vazia — e, logo abaixo, o parágrafo que diz o que a lista vazia NÃO significa.

/cmd_vel: seis publicadores do Nav2, e nenhum outro

- velocity_smoother: é ele quem leva o goal até o robô
- behavior_server, cinco vezes: spin, backup, drive_on_heading, assisted_teleop, wait
- Cada comportamento só fala durante uma recuperação — e os seis são legítimos
- A quantidade muda com os plugins do seu nav2.yaml: a regra não é por contagem
- A regra é por NOME: publicador que não é do Nav2 é defeito, e o diagnóstico o nomeia

achei. nao achei. olhe.

São os três estados, e nenhum deles é um veredito. "achei" não quer dizer certo; "nao achei" não quer dizer errado. Nos dois casos quer dizer: vá olhar isso. A ferramenta não corrige, não valida e não antecipa nota.

O que ele NÃO alcança — e é o que vale nota

- Se o robô realmente anda, e não só se os nós estão no ar
- Se o mapa publicado é do SEU cenário, e não o do exemplo
- Se a rota faz sentido na sua planta
- Se o relatório responde ao que foi pedido
- Lista vazia não é "pronto": é "esgotei o que um script enxerga"

Três casos, e o mesmo gesto para os três

- Quebrar, de propósito, no meu sistema
- Rodar o diagnóstico no Terminal B
- Ler a PRIMEIRA linha de "Onde olhar" — as outras costumam ser eco dela
- Corrigir o que ela aponta, e só isso
- Rodar de novo: a correção só vale quando a linha some
- Anotem a primeira linha de cada caso: é ela que vocês vão procurar em casa

Caso 1 — ninguém deu a pose inicial

```
# Terminal A: Ctrl-C, e de novo SEM clicar no 2D Pose Estimate
ros2 launch aula15_nav2 nav2.launch.py rviz:=true
#   planner_server: Timed out waiting for transform ... to map

# Terminal B
python3 diagnosticar_nav2.py
#   1. aresta map -> odom
```

A mensagem fala de transformada; a causa é um clique que faltou. O AMCL do Humble não publica map -> odom antes da pose, e por isso a aresta vem ANTES do nó parado: ele é eco dela.

Caso 2 — um publicador intruso em /cmd_vel

```
# Terminal C, novo (variaveis e os dois source), com o sistema no ar
ros2 run aula11_mundo piloto

# Terminal B
ros2 topic info /cmd_vel --verbose | grep "Node name"
python3 diagnosticar_nav2.py
#   nao achei  /cmd_vel so' com os publicadores do Nav2   intruso: piloto
```

Nenhum erro, em lugar nenhum: o robô obedece ao último que falou, e anda truncado. A correção é derrubar o intruso — Ctrl-C no Terminal C — e não mexer nos seis do Nav2.

Caso 3 — o gerenciador parado

```
# Terminal A: Ctrl-C, e de novo sem ativacao automatica
ros2 launch aula15_nav2 nav2.launch.py autostart:=false

# Terminal B
python3 diagnosticar_nav2.py
#   1. ativo: map_server
#      (e mais 8 nos inativos por consequencia: amcl, ...;
#      e, pelo mesmo ciclo de vida parado, a aresta map -> odom ...)
```

O gerenciador ativa um nó por vez: o primeiro inativo é onde ele parou, e os de baixo são consequência. Aqui a aresta map -> odom é consequência — no caso 1, era a raiz.

Nenhum dos três produz mensagem de erro.

É a mesma família do rviz:=true da semana passada: a primeira coisa a fazer não é ler o log, é perguntar ao sistema o que ele tem. O quarto caso frequente — o robô parado e ninguém reclamando, a cadeia do /cmd_vel_nav — está na página, com o caminho escrito.

BLOCO 2

A rota e os costmaps, demonstrados

A rota: cinco pontos, sem ninguém clicando

```
# Terminal B, com o sistema no ar e a pose dada
ros2 run aula15_nav2 rota

# a sua, com tres pontos do SEU mapa (x, y, yaw por ponto):
ros2 run aula15_nav2 rota --ros-args -p rota:="[2.0, 4.0, 0.0, 6.1, 4.0, 0.0, 9.0, 2.2, -1.57]"
```

Sem argumento, a rota padrão do mundo da Aula 11: cinco pontos, cumpridos em sequência. A sua rota usa os SEUS pontos, escolhidos sobre o costmap global — passo 5 da tarefa.

Os dois costmaps, e o que cada um faz

- Global: o mapa inteiro com a inflação, e é sobre ele que o NavFn planeja
- Local: janela rolante de poucos metros, refeita do laser a cada ciclo
- ...e é sobre ela que o DWB escolhe a próxima velocidade
- No RViz2: liguem e desliguem cada um, e a diferença fica visível
- A explicação vale tanto quanto a rota — e cabe nessas duas frases, com os SEUS números

Os quatro números do SEU YAML

```
ros2 param get /local_costmap/local_costmap width
ros2 param get /local_costmap/local_costmap resolution
ros2 param get /global_costmap/global_costmap robot_radius
ros2 param get /global_costmap/global_costmap inflation_layer.inflation_radius
```

São estes quatro que entram nos dois parágrafos, com os valores do projeto de cada um. Quem nunca abriu o próprio YAML descobre aqui o que ele diz.

Goal recusado não é goal que falhou.

Ponto em célula ocupada, ou a menos de robot_radius de uma parede, faz o servidor nem tentar. Escolha os pontos olhando o costmap global no RViz2, não o mapa cru. O robot_radius decide se a passagem abre; a inflação decide só quanto ela custa.

BLOCO 3

A árvore, e o que o Humble não tem

Os XML prontos, e os dois do exemplo

```
ls /opt/ros/humble/share/nav2_bt_navigator/behavior_trees/
cd "$PB_DIR/aula16-arvore-bag" && ls arvores/

# padrao_comentada.xml  -- a oficial, comentada bloco a bloco
# com_espera.xml        -- a variante com a espera de 2 s na recuperacao
```

O bt_navigator executa um XML: um arquivo de texto, que dá para abrir e ler. Copie para o SEU pacote e aponte pelo parâmetro do bt_navigator, ou pelo campo behavior_tree do goal — que o rota.py já sabe preencher, com arvore:=.

No Humble, não há nó de árvore que meça distância a obstáculo.

Conferido na lista oficial de nós registrados: há condição para bateria baixa, para objetivo trocado, para caminho inválido, para tempo decorrido — e nenhuma para "obstáculo a menos de 0,5 m". A árvore sozinha NÃO cumpre o enunciado.

Então quem diz que há obstáculo é você, por um de dois caminhos

- Completo: um nó de condição próprio, em C++, plugin do nav2_behavior_tree...
- ...composto com o Wait de 2 s dentro de um ReactiveFallback
- De cliente: o vigia.py assina o /scan, cancela o goal, espera 2 s e reenvia
- O segundo é a Aula 8 de novo — cancelamento de ação — e não compila C++
- Os dois passam. O que não passa é não dizer qual você usou, e por quê

Duas armadilhas que não dizem o nome

- O wait_duration do nó Wait é INTEIRO: escrever 2.0 faz a árvore não carregar
- E o bt_navigator falha ao ativar sem apontar a linha
- Comentário de XML não pode conter dois traços seguidos
- Toda opção do ROS 2 começa com dois traços: colar --ros-args num <!-- --> ...
- ...faz o parser recusar o ARQUIVO INTEIRO, com mensagem que não fala de traços

BLOCO 4

A sessão gravada, e a figura

Gravar, conferir, tocar de volta

```
# Terminal C, com o sistema no ar e um goal andando: 20 s, e Ctrl-C
ros2 bag record -o demo-aula16 /odom /amcl_pose /scan /cmd_vel /tf /tf_static

ros2 bag info demo-aula16          # /tf_static: UMA mensagem

# depois, com TUDO desligado:
ros2 bag play demo-aula16 --clock
```

O /tf_static sai UMA vez, no instante em que o sistema sobe. Fora da lista do record, a reprodução é um laser flutuando sem corpo — e você descobre depois, quando não há mais como regravar.

A régua antes do robô: a sessão sintética

```
cd "$PB_DIR/aula16-arvore-bag"
python3 simular_sessao.py                       # gera sessao.csv
python3 medir_sessao.py                         # a tabela
python3 plotar_trajetoria.py --saida trajetoria.png
```

Determinística, com dois eventos de obstáculo. A figura e a tabela saem sem robô nenhum — e a MESMA régua mede o bag de vocês quando ele existir.

A tabela, e a primeira linha é a aula

```
DISTANCIA PERCORRIDA -- tres contas, tres numeros
  integral de |cmd_v| dt   :  25.52 m   <- o que foi mandado andar
  soma dos passos do /odom :  26.29 m   <- +3.0% (a deriva de escala)
  soma dos passos do /amcl :  49.03 m   <- +92.1%: o tremor virou caminho

INTERRUPCOES -- 2 evento(s); o portao viu no minimo 0.402 m
  1   t=12.30 s   parado 2.00 s   espera 2.00 s   2 s cumpridos: sim
  2   t=51.80 s   parado 2.10 s   espera 2.10 s   2 s cumpridos: sim
```

O robô andou 25 m e a trajetória do /amcl_pose tem 49. Não é defeito do AMCL: é a conta que está errada.

Somar passos de uma estimativa que treme soma o tremor.

Noventa e dois por cento de inflação saem de um ruído de poucos centímetros. Quem citar "distância percorrida" a partir do /amcl_pose está citando ruído — e esta é a terceira vez no semestre que uma métrica mede o instrumento, não o mundo.

As outras duas leituras da tabela

- O vão entre as duas trajetórias é 0,75 m, e ele tem nome
- Não é "erro do AMCL": é o que a transformada map -> odom corrige
- É a deriva acumulada da odometria — escreva assim no relatório
- E as duas esperas são 2,00 s e 2,10 s, diferentes de propósito
- Tolerância declarada antes de medir: 2,00 s ± 0,25 s. Como na Aula 9

A tarefa da semana: é ali que a aula acontece para você

- 1. Preparar os terminais: as três variáveis e os dois source, em cada terminal novo
- 2. Subir o sistema, dar a pose, rodar o diagnóstico e LER a lista
- 3. Se sobrar linha: os três casos de hoje, e o quarto — cada um com o que fazer
- 4. O goal no SEU mapa, e os 20 s de vídeo do robô chegando
- 5. A rota de três pontos, e os dois parágrafos dos costmaps com os SEUS números
- 6. A escolha do caminho da interrupção, em três frases
- 7. Como saber que terminou: a lista está no fim da tarefa

Como saber que terminou

```
ros2 action list | grep navigate              # navigate_to_pose
ros2 topic info /cmd_vel --verbose | grep "Node name"
#   so velocity_smoother e behavior_server
ros2 run tf2_ros tf2_echo map odom            # existe, e NAO e identidade
```

E o diagnóstico sem pendências — com o parágrafo do que ele não alcança lido. Travou e nada na página bate? Fórum do Moodle, com a saída do diagnóstico e o passo em que você está.

Desafio — medir a sessão antes de ter uma

- Rode a régua no CSV sintético e leia a tabela inteira
- Depois mude UMA coisa: --sem-eventos, uma espera de 1,2 s, outra semente
- Qual coluna da tabela denuncia a mudança? E qual fica igual?
- Quem responder isso já sabe o que procurar no próprio bag

Na próxima: a sua escolha, funcionando ou travada.

Quem for de C++ traz o plugin compilando; quem for de cliente traz o vigia rodando. Começam também os itens 3 e 4, e neles a parte que não comprime são os 500 frames rotulados: comecem a salvar frames esta semana. As datas estão todas na página da aula e em "Gates de cada TP".